CSC7062 – Christopher Barber - Web Development Project Written Report

Introduction

The purpose of this report is to support the Web application, HedClass, which has been developed and implemented in line with the CSC7062 module assessment brief. This report will describe the overall system and its architecture. It will then discuss in greater detail the features and how these meet the specification of requirements. Finally, the report will demonstrate how the code is structured and how it was developed.

Overview

The purpose of the web application, HedClass, is to offer a student award classification platform for Higher Education Institutes. The central feature of the system enables end users called 'Officers' the opportunity to log into the system securely, input information about a student's performance, calculate (using automated rules) a student's qualification classification and view general management information about courses. The second major aspect to the system is to enable 'Admin' users to log into the system and administrate Officers and Courses i.e. in both cases add new Officers/Courses and/or edit details about Officers.

The aim of the system is to improve the process of course qualification classification for students. Currently the process is completed manually. This can lead to inconsistent application of rules, inaccuracies around data as well as scalability issues as student cohort's grow. The system then, implements features to reduce manual errors, increase consistency of classification accuracy, save manual administration time (as student numbers grow) as well as improve auditability of the process.

The application is built using 3 layers. First is the Presentation layer. This is the interface for end users – what they see and can interact with. This implementation uses templated HTML, styled using (primarily) a CSS framework along with some custom CSS. To produce the specific content for each page/route, embedded JavaScript was used in order to dynamically populate with data from the database layer to the front end. Secondly the Application layer was built using JavaScript and specifically using Node.js as the server-side runtime environment and leveraging the JavaScript framework, Express (and the abstraction it brings) in order to easily and cleanly handle HTTP requests. The application layer handles all HTTP routing requests as well as setting out specific business logic (e.g. classification calculation), database updates and some custom middleware. Finally, there is the Data layer. All of the data relating to students, courses, system users etc. is stored in a persistent database. The database

technology used was the relational database platform, MySQL. The Node package mysql2 was used in order to get the Application layer to interact with the Data layer.

Functional Requirements Breakdown

What follows is a step-by-step walkthrough of the system, highlighting navigation and functionality and linking each step back to the specification of requirements provided as part of the assessment brief.

When first accessing the application the end user must login to the system using an email address and password (Requirement A1 – see Fig 1). Login credentials for end users are stored in the accompanying database and the application attempts to match the inputted values vs. stored records. End users are also assigned different role types (Officer and Admin) when they are first set up. For the benefit of the development phase there was also an Officer (view only) access level created but this isn't specifically part of the specification of requirements. Each of these role types provide different levels of access control and this level of access is again stored securely in the accompanying database. Each route and corresponding page of the application is protected with a call to the database to check whether the access level that has been granted upon login matches the requirements of that page/route. A simple example to demonstrate this in this report, is the role-based differences in available navigation options within the main navigation Header (Fig 2.1 + 2.2). Depending upon the end user's level of access, more or less buttons can be seen, but the Header in its entirety is demonstrated in Fig 2.1.

In order to preserve state when navigating through the application and to ensure authentication, the express-session framework has been implemented (Requirement A2 – Fig 3).

Although there are several sub features of the application, these have been overarchingly grouped into the 3 sections seen in Fig 2.1 – Dashboard, Student Management and Course Management.

Within 'Dashboard', Officers can view a high-level summary of all courses, including the number of students aligned to the course, average score achieved of those students and total number of the types of classification awarded (Requirement C1 – see Fig 4). This section also includes statistical bar charts, which provide a visual aid in demonstrating the distribution of awarded classifications (Requirement C6 – Fig 4). On this screen there is also the option to export the displayed dashboard data into a csv file. This enables Officers to be able to open the data in e.g. Excel, manipulate the data further and then share the total data about courses, numbers of students and performance by course with management teams i.e. the board of examiners (Requirement C7 – Fig 4).

‘Student Management’ is the next critical section. Starting here Officers can see Student records for each of the students that are enrolled on a Course the Officer manages. Officers can also edit/delete the Student record or indeed add new students (Requirement C2 – Fig 5). From within each Student record, Officers can also view more detailed information about the student’s academic performance. Officers can select ‘View Detailed Student Results Information’ which will then enable them to see detailed information at a Module level – i.e. what modules within the course they are enrolled on and what result they have achieved. Officers can add new Module result and even more importantly, this is the screen where Officers can request the application to ‘auto calculate’ what classification the specified student has achieved (Requirement C3 – Fig 6). In order for the calculation to execute, the student must have a full complement of 17 module results inputted against their record. These 17 modules cover all 3 years of academic evaluation (6 modules in y1 + y2 and 5 modules y3 (including double credit dissertation module)). The classification calculation is based on the standard criteria, as given in the specification of requirements. There is the option to adjust the calculation parameters - but that will be covered in more detail later in the report. At this stage, if a classification has been awarded then a button will appear on this screen with the title ‘Manually Override Classification’. When an Officer clicks this button, they are brought though to a new screen where they can manually override what the student’s new classification is (Requirement C4 – Fig 7). This will overwrite the previous record, mark the new award as a manual override as well as give them the opportunity to input what the qualitative rationale for the manual classification was (Requirement C5 – Fig 7).

‘Course Management’ is the next major section. This section of the application is where new courses can be added and edited. The creation of new courses is part of the Admin role functionality - so that will be covered in more detail later in this report. However, the ability to edit a course specifically is part of the Officer role functionality. From this screen, Officers can select individual courses and edit the classification award criteria for that course. Several variables can be adjusted – e.g. the weighting of y2/y3 results, the max award when a Student completes a resit, as well as the thresholds for the level of award i.e. the boundaries for when a final score equals a third, 2:1, 1st etc. This means that individual courses can vary their classification award criteria rules - adopting either the standard Institute level variables or course specific variables (Requirement C8 + C9 – Fig 8).

The final major section of the application, ‘Officer Management’ is primarily for end users with the Admin role. Here Admins can view all current Officers as well as edit / delete their records (Requirement B1 – Fig 9). When adding a new officer, as well as inputting basic administration information about the Officer, they can also assign the Officer to a specific course to manage (Requirement B2 – Fig 10). When editing an Officer record, again, as well as editing their basic personal information, there is a

chance to update the course that the Officer manages. Within this screen, the Admin user can select one *or* multiple courses for the Officer to manage (Requirement B4 – Fig 11). This is important as it dictates the information that the Officer role can see when they log in, i.e. they will only see course information and student information that is relevant to the courses that they manage.

From an Officer perspective, the other major functionality that they have access to, is to create a new course. From this screen, the Admin user only needs to enter the title of the course and then select 'Add Course'. When this command is executed, the course record is automatically assigned the standard classification calculation variables that have been defined in the specification of requirement (Requirement B3 – Fig 12) before being added to the table of courses in the database.

Non-Functional Requirements Breakdown

The application supports scalability on two levels (Requirement D1). Firstly, the ability to auto calculate the classification of a student's award means that no matter how many students there are, the calculation of the award is an extremely quick process to execute. Secondly the ability to create new students, courses and results means that expansion in any of these areas can be again executed very quickly. Further, with the ability to tweak the classification rules of each course, if any Institute adopting the system were to grow or evolve in complexity, the complex nature of the classification calculation is always going to be able to be applied quickly and consistently (Requirement D2).

With regards to security measures, the application manages this in three ways. Firstly, with Authentication - in order to gain access to the system valid end user credentials must be provided (email / password). Please note although for the purposes of this demonstration passwords are stored in the database as plain text, in the real implementation, any passwords would be encrypted using a hashing algorithm. Secondly, Authorisation has been implemented – i.e. different areas of the application are only accessible depending upon what type of role the end user has. Each route throughout the application checks both whether the end user is authenticated and then whether their role is authorised to view or edit the content of that end point. Lastly, for the critical areas of classification award, when changes are made e.g. manual classification of an award, the Systemuser ID which is unique to each end user is captured and logged into the updated record. This means there is a clear audit trail as to who has awarded a student a classification (Requirement D6). Further, if an Officer (who has the power to award a classification) were to be deleted, the application makes sure to, before deleting their record (and therefore removing the Systemuser ID from the award record), update the award with the actual name of the Officer who made the award. All of these aspects support meeting Requirement D3.

Aesthetic / styling

In order to build the front end of the system, an evaluation was undertaken to consider creating CSS from scratch or to leverage an existing CSS framework. On balance, it was felt that the time required to read/understand the necessary documentation and manipulate pre-built code outweighed the work required to produce consistent working vanilla CSS to the same standard. Once the decision was made to use a CSS framework, a brief review of potential options was also undertaken. More heavyweight frameworks were considered such as Bootstrap however ultimately the more simple, light touch framework of Mini CSS was chosen to provide the bulk of the styling required, which whilst being relatively simple and easy to understand, still enabled a modern and responsive user interface design (Requirement D-4 – Fig 13). This was however also supported by a small amount of bespoke, self-written CSS in order to tweak certain aspects of the framework where required. E.g. to override the style of a Form which was predefined within Mini CSS.

For system confirmation or error messages e.g. where a new student has been added or a classification has been edited, 'Toast' messages were used to confirm the key information back to the end user.

Code structure / System Architecture / API

The infrastructure of the codebase follows a simple model as can be seen from Fig 14, whereby under the main web folder there are folders for:

- Middleware – This contains db.js which constitutes the code required to connect to the MySQL database. This also contains services.js, which is the home for the checkAuth middleware function (frequently repeated check on login credentials vs. role required for access). Other middleware, if created, would also sit here.
- Routes – Within this section, there are three route files. The first is admin routes. Here code reviewers would find all routes associated with the 'Admin' role. Secondly there is edit routes. Here code reviewers would find all routes associated with the 'Officer' role. Lastly, there is view routes. Contained here are generic routes such as login, error and the initial landing page.
- Views – This section is split into further sub folders and contains all of the embedded JavaScript (.ejs) files. The sections very much follows the Routes section in that there are admin, officer and standard folders.

The main app.js was initially built as a monolithic application. However, to improve readability and maintainability this file was split up into more manageable chunks – leading to the separation across the three different routes folders. This leaves the app.js file as a very simple shell file that sets up the express server and handles initial login and error routes.

This structure helps improve readability and maintainability, for any future maintainer of the code. However, it is recognised that this aspect could be improved further through the adoption of the full Model View Controller architecture, whereby routes are broken down further, leading to functions that interact with the database (Model), functions that perform the business logic (Controller) and files that handle what the end user sees (View). Although the ejs files within the Views folder are well separated and contain no business logic or database calls, the routes that have been created contain both database access functions and business logic functions.

Throughout the project, the version control system GitLab was used (requirement D:5 – Fig 15). Where possible atomic updates were made i.e. where each commit constituted a single defining change. There were however some commits that were collections of small changes e.g. small bug fixes or aesthetic tweaks.

Direct access to the MySQL database was used throughout the application, as explained in the Overview section. Incrementally, an API has also been created in order to act as a gateway to the MySQL database (Requirement E2 – Fig 16). Although this API is only partially complete, it does demonstrate that the database can be accessed through a URL endpoint rather than with direct query language. The server.js file demonstrates GET requests (read) to: all students, specified students, all courses and specified courses. It also enables POST requests (create new) to the systemUsers table. It also enables PATCH requests (update) to systemUsers table. All of these routes have been tested and are working using the third-party software, Thunder Client. Again, the API has not been adopted throughout the application however there is a test page to further demonstrate that a. the API works on one of the real, in-application routes and b. can be rendered into one of the standard. ejs templates. This page can be found at http://localhost:3000/apitest.

Appendix

Fig 1 – Login Window

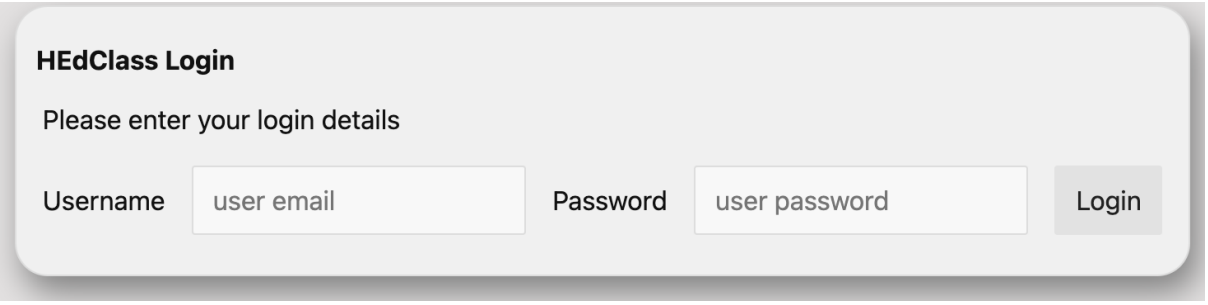


Fig 2.1 – Variation in Header visibility depending upon role

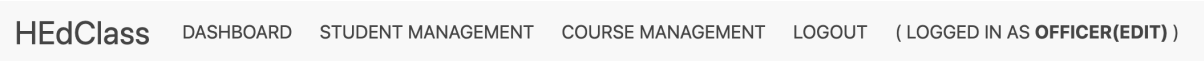


Fig 2.2 – Variation in Header visibility depending upon role

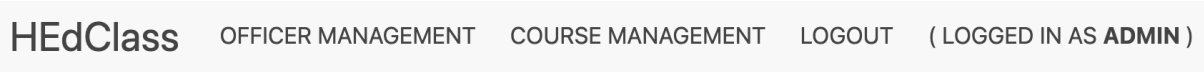


Fig 3 – State Management – evidence of unique session ID being created once logged in

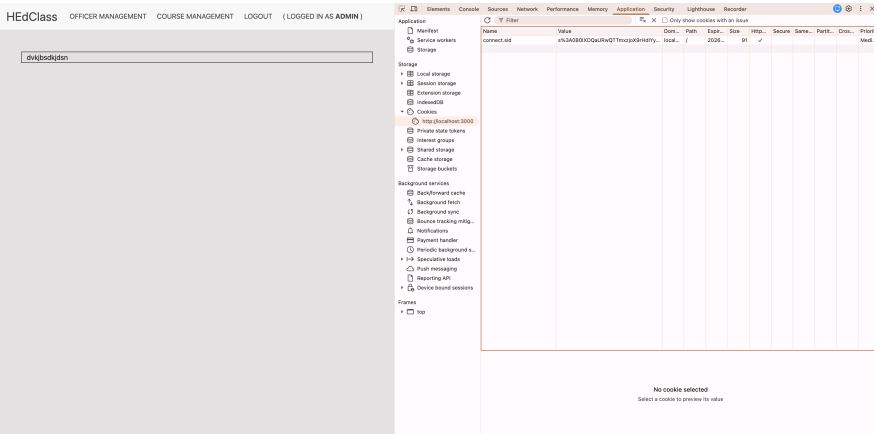


Fig 4 – Dashboard – Requirement C1 + C6 + C7

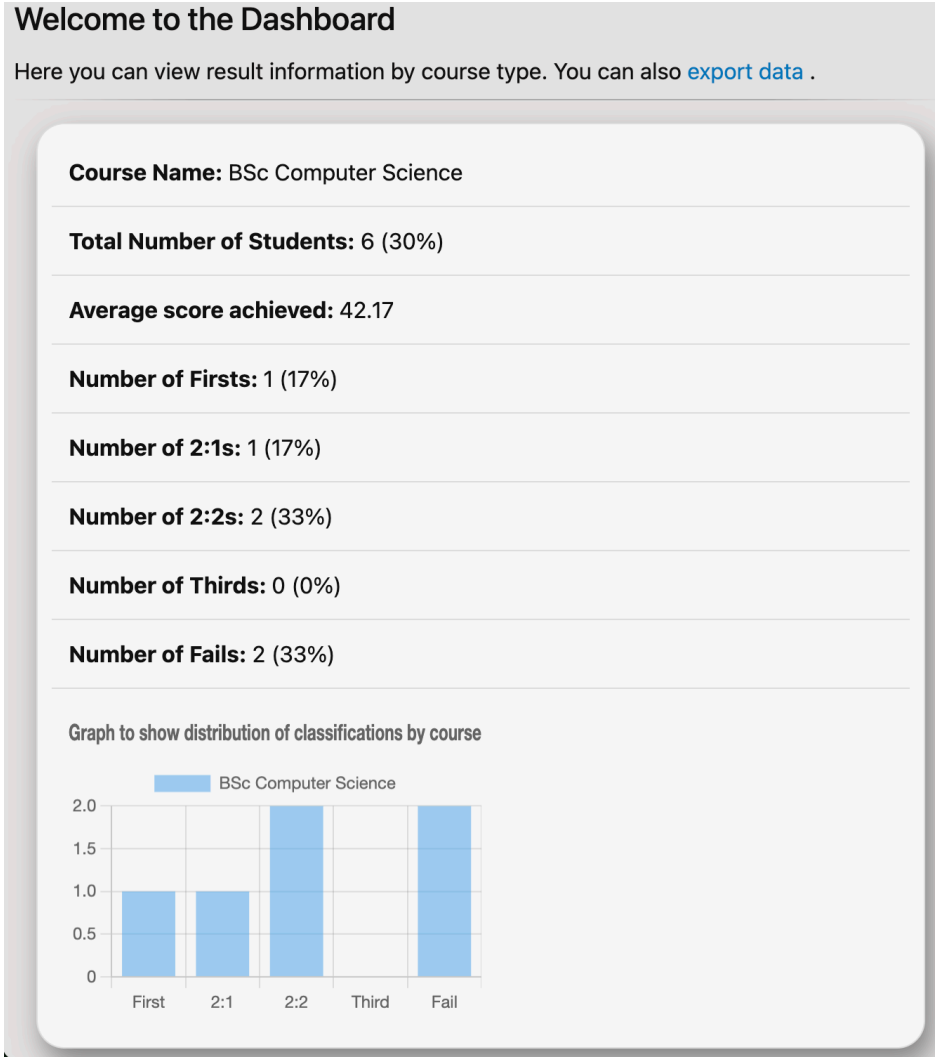


Fig 5 – C2 – Student CRUD

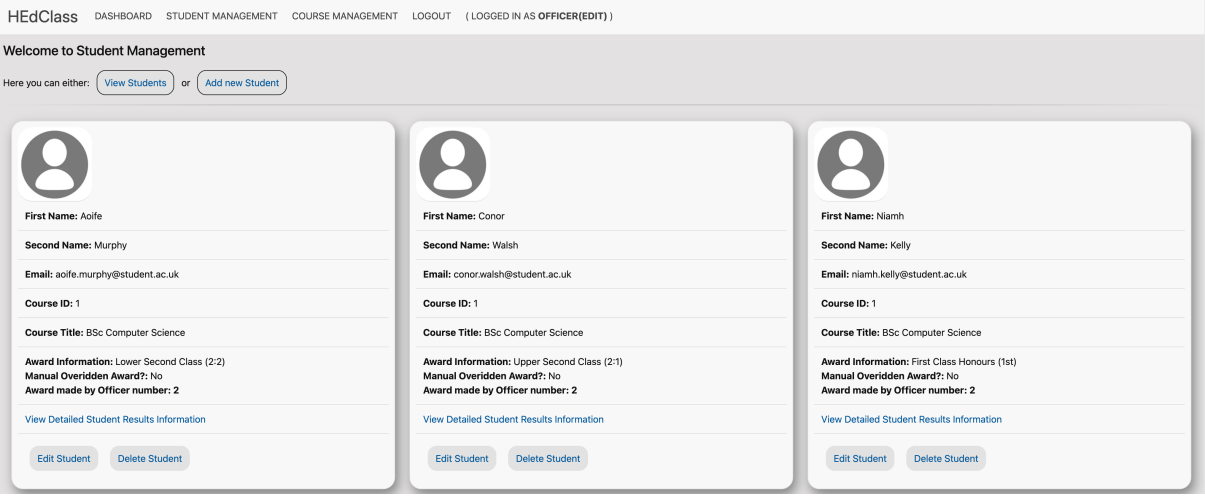


Fig 6 – Auto calculation of Classification

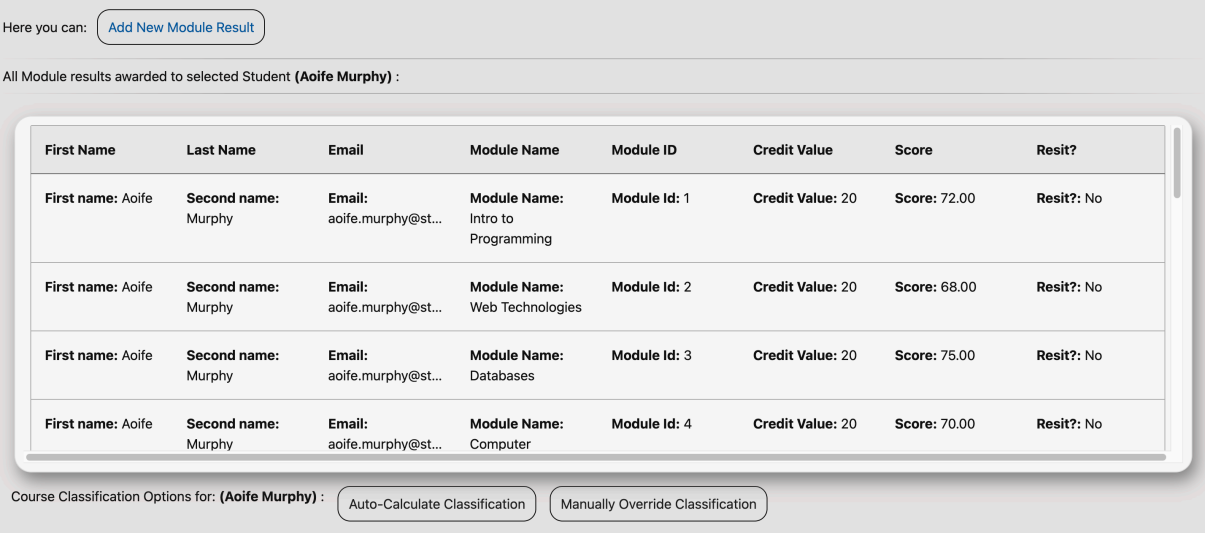


Fig 7 – Manul Override of Classification and decision rationale input

Fig 8 – Edit classification award rules

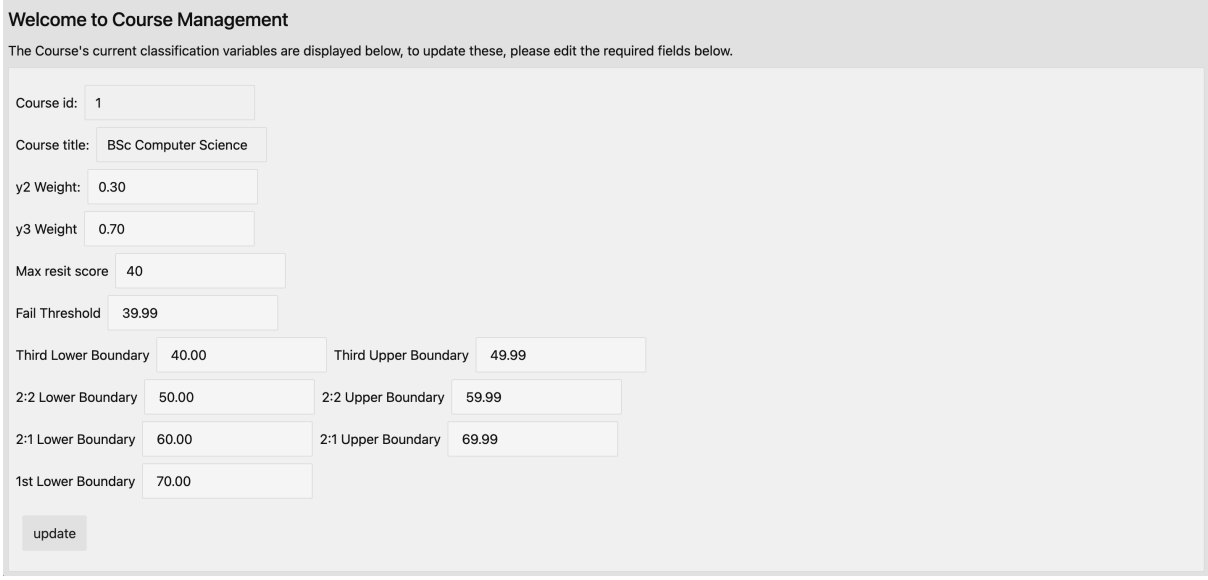


Fig 9 – Create/Manage Officers

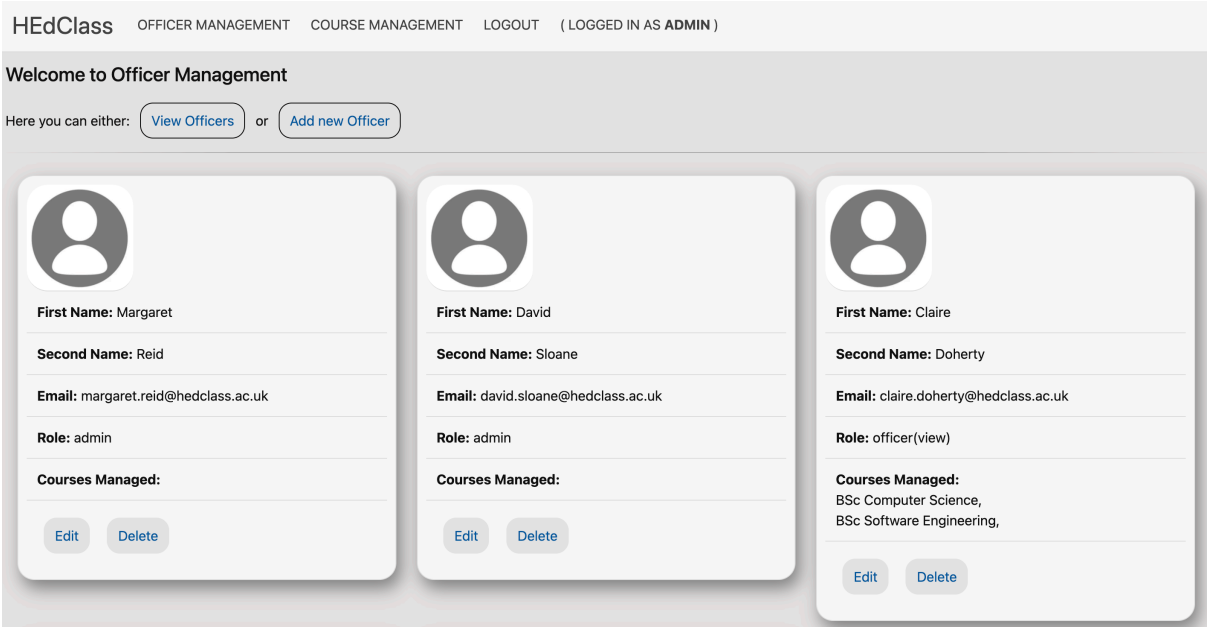


Fig 10 – Assigning new Officer to a specific course

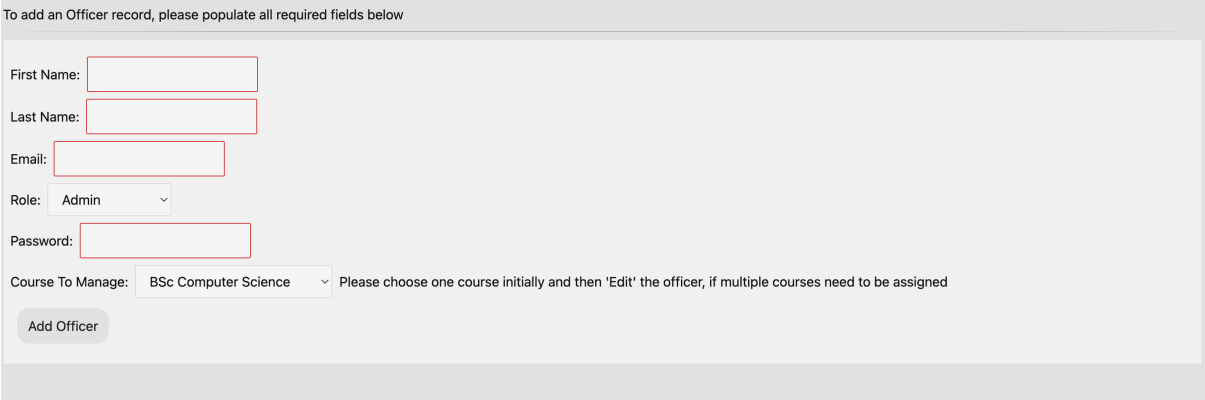


Fig 11 – Updating Officer details, including assigning the Officer to more than one course

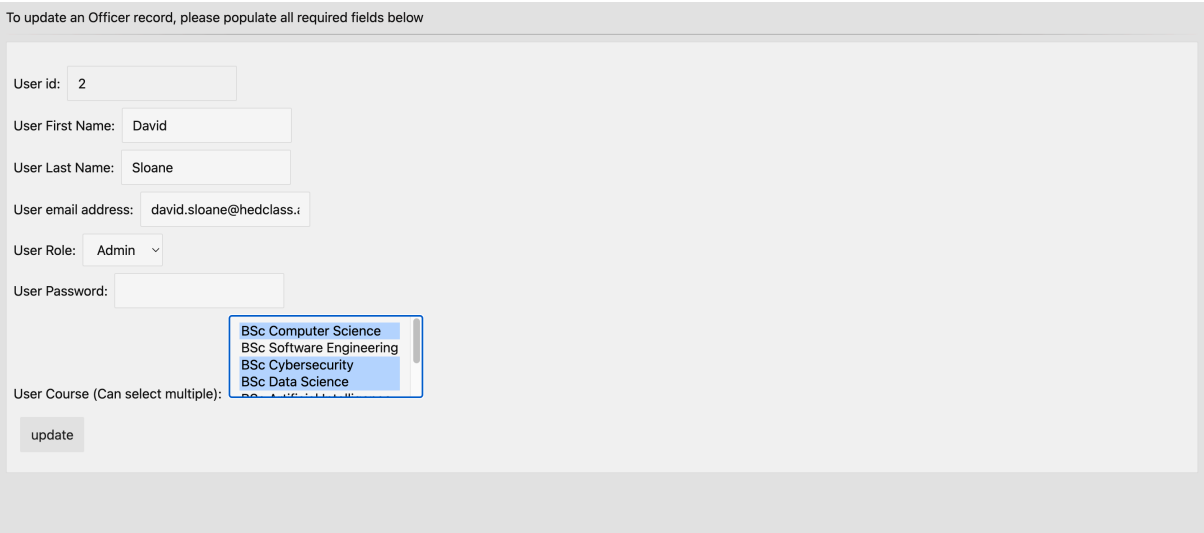


Fig 12 – Creating a new course with the standard classification calculation variables

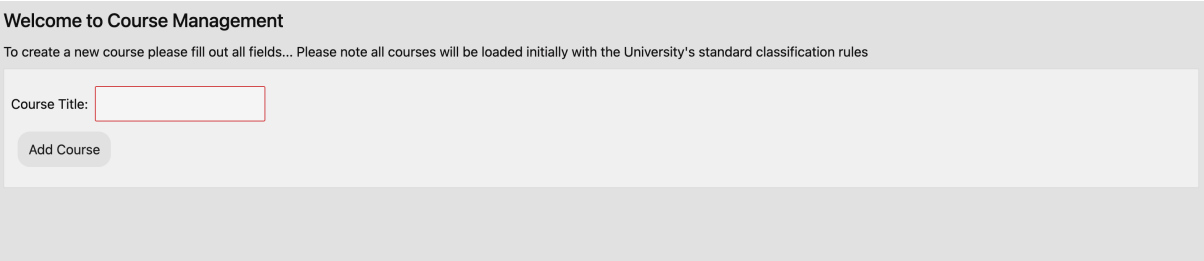
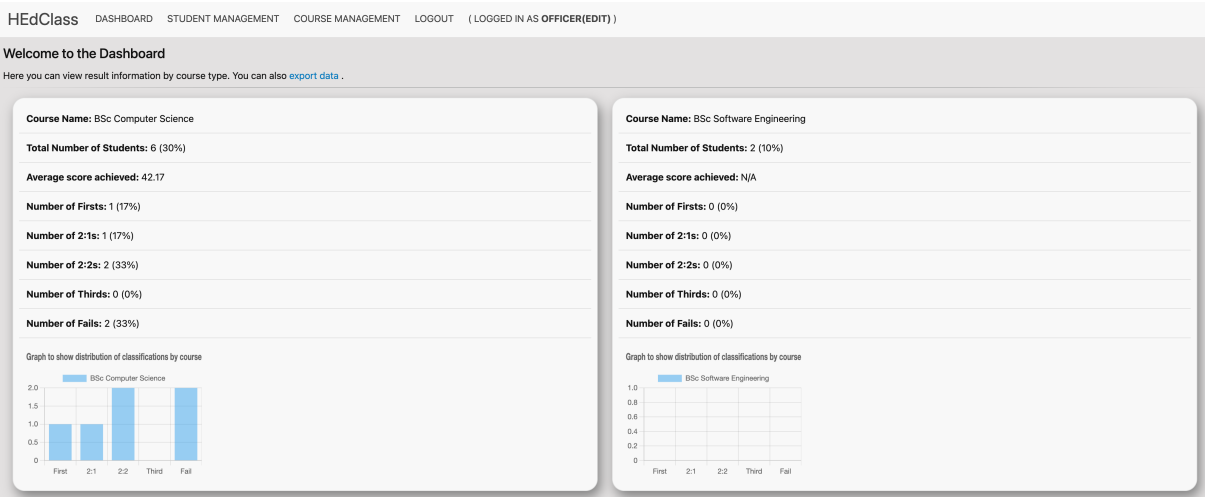
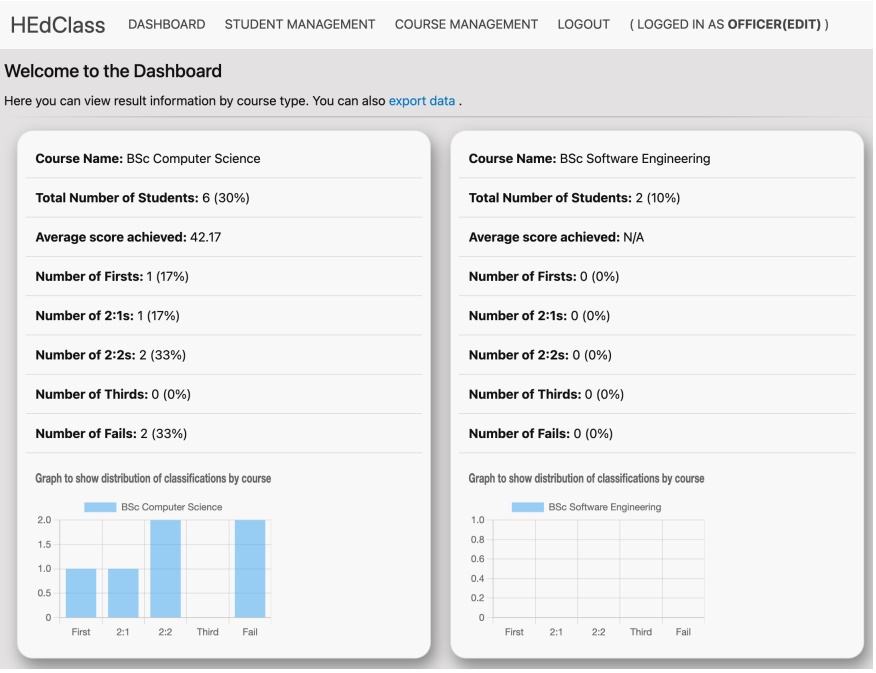


Fig 13.1, 13.2 and 13.3 – Demonstration of responsive design as browser window shrinks across Large, Medium and Small

13.1 - Large



13.2 - Medium



13.3 - Small

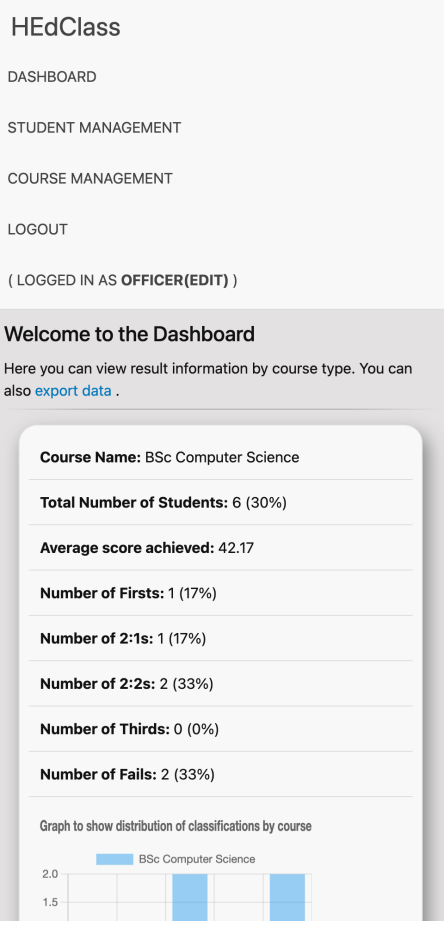
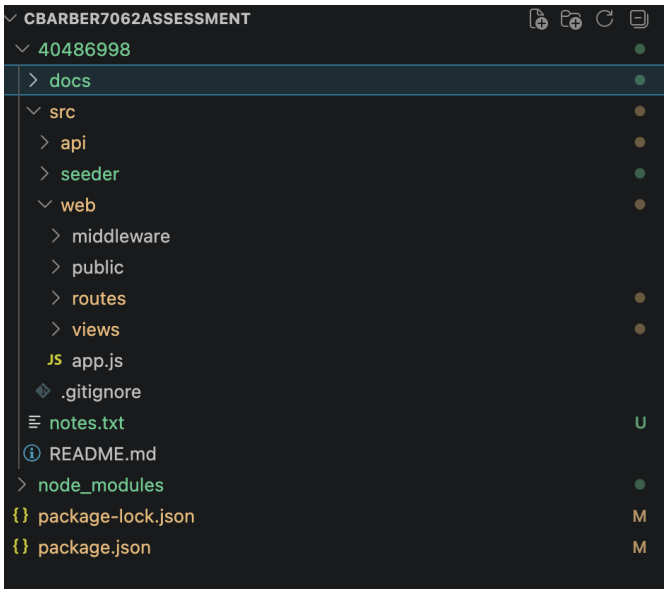


Fig 14.1 + 14.2 – Code Structure Hierarchy



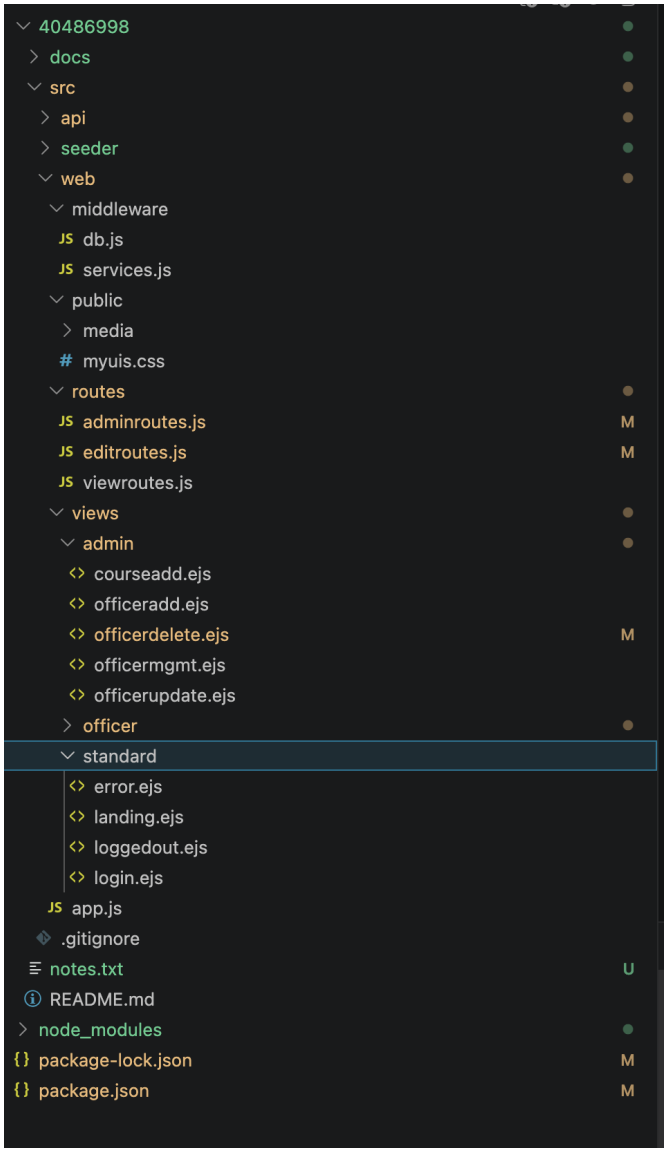
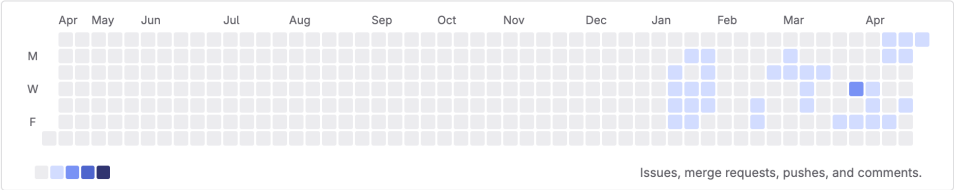

40486998
docs
src
api
seeder
web
middleware
db.js
services.js
public
media
myuis.css
routes
adminroutes.js M
editroutes.js M
viewroutes.js
views
admin
courseadd.ejs
officeradd.ejs
officerdelete.ejs M
officermgmt.ejs
officerupdate.ejs
officer
standard
error.ejs
landing.ejs
loggedout.ejs
login.ejs
app.js
.gitignore
notes.txt U
README.md
node_modules
package-lock.json M
package.json M

Fig 15 – GitLab Usage

Activity

View all



Pushed to branch `main` at 40486998 / cbarber7062assessment — just now
`b5f44314` · added API routes. Added apittest page

Pushed to branch `main` at 40486998 / cbarber7062assessment — 2 days ago
`05cc8631` · final bug fixes and added basic server API

Pushed to branch `main` at 40486998 / cbarber7062assessment — 2 days ago
`203ed7c3` · added edit course functionality

Pushed to branch `main` at 40486998 / cbarber7062assessment — 3 days ago
`acf73edc` · added filewriter to enable export of dashbaord data

Pushed to branch `main` at 40486998 / cbarber7062assessment — 3 days ago
`bf175716` · updated add result page to help with inputting results more easily

Pushed to branch `main` at 40486998 / cbarber7062assessment — 3 days ago
`a0e8ac87` · added export link

Pushed to branch `main` at 40486998 / cbarber7062assessment — 3 days ago
`b43856e9` · small aesthetic tweaks including updating header on edit classifica...

Pushed to branch `main` at 40486998 / cbarber7062assessment — 5 days ago
`88ba6f55` · enabled cusotm course rules ot be pulled through for calculation as...

Pushed to branch `main` at 40486998 / cbarber7062assessment — 5 days ago
`d2923d1c` · completely redefined app structure - sorted middleware, views and r...

Pushed to branch `main` at 40486998 / cbarber7062assessment — 6 days ago
`2c61462a` · finalised CSS, updated add result as new page

Pushed to branch `main` at 40486998 / cbarber7062assessment — 6 days ago

Fig 16 – API (GET + POST + PATCH)

GET http://localhost:4000/students Send

Status: 200 OK Size: 3.17 KB Time: 364 ms

Response Headers 6 Cookies Results Docs

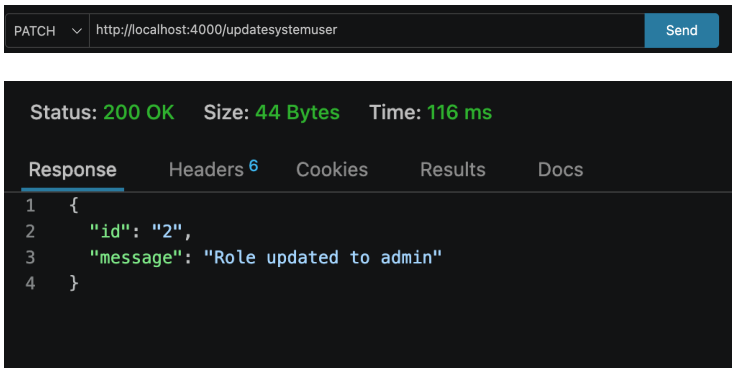

```
1  {
2    "data": [
3      {
4        "id": 1,
5        "firstName": "Aoife",
6        "lastName": "Murphy",
7        "email": "aoife.murphy@student.ac.uk",
8        "courseID": 1,
9        "graduationYear": "25/26",
10       "awardID": 1
11     },
12     {
13       "id": 2,
14       "firstName": "Conor",
15       "lastName": "Walsh",
16       "email": "conor.walsh@student.ac.uk",
17       "courseID": 1,
18       "graduationYear": "25/26",
19       "awardID": 2
20     },
21     {
22       "id": 3,
23       "firstName": "Sophie",
24       "lastName": "Clarke",
25       "email": "sophie.clarke@student.ac.uk",
26       "courseID": 1,
27       "graduationYear": "25/26",
28       "awardID": 3
29     },
30     {
31       "id": 4,
32       "firstName": "Liam",
```


POST http://localhost:4000/students/add Send

Status: 200 OK Size: 55 Bytes Time: 288 ms

Response Headers 6 Cookies Results Docs


```
1  {
2    "id": 31,
3    "message": "Christest added to student table."
4  }
```



Important Links and Login Credentials

- Repo address: https://gitlab.eeecs.qub.ac.uk/40486998/cbarber7062assessment
- Login for officer role: officer@test.com (officer(edit)). Password: admin
- Login for admin role: admin@test.com (admin). Password: admin

References

Please note to support the generation of the test data for the database, the use of the LLM AI Model Claude was used. The reason for this use was that once the database schema had been defined, to speed up the manual process of creating test documents, which was a low value activity, this was best performed by an automated tool. Please see below example prompt, where I use my existing defined database schema and requested test data that can be used with it. Please note during development I also increased the size of the data seed in order to better demonstrate the system (e.g. I have 20 students now instead of10).

based on this final schema design, please can you create me a sample set of data that I can seed: 10 students, 5 courses, 3 officers, 2 admins

Viewed a file, created a file, read a file >

Here's your seed file. Here's a summary of what's included:

SystemUsers — 2 admins (Margaret, David) and 3 officers (Claire, James, Niamh), each assigned to their courses via ManagedCourses.

Students — 10 students spread across all 5 courses, each with a deliberate classification profile:

Student	Course	Target Classification
Aoife Murphy	BSc Computer Science	1st
Conor Walsh	BSc Computer Science	2:1
Sophie Clarke	BSc Software Engineering	2:2
Liam O'Brien	BSc Software Engineering	3rd (Y2 resit)
Emma Stewart	BSc Cybersecurity	1st
Ryan Brennan	BSc Cybersecurity	Not Eligible (Y1 fails)
Chloe Hughes	BSc Data Science	2:1
Nathan Fox	BSc Data Science	2:2 (Y3 resit)
Isla Campbell	BSc AI	1st
Patrick Doyle	BSc AI	3rd